

Mid-Point Summary Report

1. Executive Summary

The first four weeks of the internship established a complete evidence chain from strategic orientation to technical evaluation. Phase A focused on understanding the embodied AI landscape, InGen's product portfolio, and the broader competitive context. Phase B translated that context into a concrete telemetry analytics and anomaly detection workflow using Aido Rover as the main telemetry substrate and Sentinel-style fleet monitoring as the operational framing.

By the end of Week 4, the work had moved beyond general research and into a reproducible applied analytics pipeline. The main outputs include a physical AI landscape brief, PIC 2.0 conceptual map, competitive vendor matrix, strategic gap memo, synthetic Rover telemetry generator, EDA notebook, feature dictionary, anomaly detection benchmark, method recommendation memo, and mid-point summary deck. The Week 4 benchmark tested four anomaly detection methods across 10 seeded train/test splits, using five controlled fault patterns and paired Wilcoxon tests for AUROC comparisons. The current recommendation is to use LOF as the primary accuracy path, keep One-Class SVM as a close challenger, and use Isolation Forest only as a lightweight first-pass screen with rule-based guardrails.

Overall, the strongest part of Phases A-B is the connection between product context and quantitative method evaluation. The work did not stay at a generic machine-learning level. Each step was tied back to a practical fleet-health question: what kind of anomaly detection approach would be useful for Rover and Sentinel-style monitoring, under realistic constraints around latency, interpretability, false alerts, and reproducibility?

2. Activities Completed

Week 1: Landscape Orientation and Toolchain Setup

Week 1 focused on building enough domain literacy to make later technical work meaningful. I reviewed the physical AI and embodied intelligence landscape, including vision-language-action models, humanoid foundation models, hierarchical reinforcement learning, sim-to-real transfer, and robotics industry trends. I also created a PIC 2.0 conceptual map to connect InGen's foundation-model framing with open-literature equivalents such as policy optimization, uncertainty calibration, safety constraints, adaptive control, hierarchical task decomposition, and multi-robot learning.

The other important Week 1 outcome was environment setup. I created a Python 3.11 toolchain with the libraries needed for the later weeks, including pandas, NumPy, scikit-learn, PyTorch, statsmodels, Gymnasium, Stable-Baselines3, PettingZoo, and Streamlit. This gave the repository a reproducible technical base instead of treating each deliverable as a one-off file.

Week 2: Competitive Intelligence and Strategic Gap Analysis

Week 2 moved from general landscape research to a more structured competitive view. I built a vendor matrix covering humanoid, service robot, and outdoor patrol/security categories, with each vendor evaluated across positioning, hardware, verticals, technical direction, funding or market stage, and recent public milestones. I also cross-referenced InGen's platforms against comparable vendors and product categories.

The strategic gap memo distilled the competitive research into three main observations. First, many competitors appear focused on one embodiment or one vertical, while InGen can position itself around a shared intelligence layer across multiple embodiments. Second, human-centered adoption areas such as care, education, and companion use cases require trust, safety, and decision quality, not only impressive robot hardware. Third, security and patrol systems need anomaly logic that is governance-aware, auditable, and designed around false-alert control. These ideas became important context for the Week 4 anomaly benchmark.

Week 3: Synthetic Telemetry Generation, EDA, and Feature Engineering

Week 3 was the transition from research to data work. I implemented a parameterized synthetic telemetry generator for Aido Rover-style fleet monitoring. The generator creates 1 Hz telemetry with signals such as joint and actuator angles, IMU acceleration and gyro channels, motor current per drive, battery voltage and state of charge, WiFi RSSI, GPS fix quality, and task-success flags. The default sample used for EDA and Week 4 testing contains 432,000 rows from 5 Rover units over 1 day, while the generator can scale to a 50-unit, 30-day configuration.

I then completed an EDA notebook covering stationarity testing, STL seasonality decomposition, correlation structure, and missingness patterns. The feature dictionary documented 25 engineered features, including short-window rolling statistics, cross-sensor consistency ratios, and inter-unit z-scores. This was an important design choice because the anomaly benchmark uses features with physical meaning, rather than treating the telemetry as a generic table of numbers.

Week 4: Fault Injection, Anomaly Benchmark, Recommendation, and Review Deck

Week 4 built the anomaly detection benchmark. I injected five controlled fault patterns into the Week 3 dataset: motor stall, gradual sensor drift, accelerated battery degradation, GPS jitter with spoofing-like behavior, and intermittent software hang. The benchmark included leakage controls so that faults are injected only after the train/test split and only into held-out blocks. Scalers are fit only on clean training windows, and all methods are compared across the same 10 seeded splits.

The four methods tested were Isolation Forest, One-Class SVM, Local Outlier Factor, and an LSTM autoencoder. I evaluated precision, recall, F1, and AUROC overall and by fault type, then ran paired Wilcoxon signed-rank tests on AUROC differences. Based on the benchmark, LOF is currently the preferred method for accuracy-maximizing analytics and fleet-health triage. One-Class SVM remains a close challenger, especially because its overall AUROC was close to LOF. Isolation Forest is still useful for a latency-constrained first-pass screen, but its lower AUROC means it should not be the only decision layer for higher-risk alerts. The LSTM autoencoder is conceptually useful, but in this benchmark it did not justify its added runtime and tuning complexity.

3. Outcomes Achieved

The most important outcome is that Phases A-B now form a coherent applied analytics pipeline. The work started with product and market understanding, moved into telemetry design and feature engineering, and ended with a statistically tested anomaly detection recommendation. This matters because the final internship deliverables should not look like disconnected weekly exercises. They should tell a clear story about how competitive intelligence, telemetry analytics, anomaly detection, decision evaluation, and dashboard design can support an applied AI product organization.

A second outcome is stronger evidence discipline. The Week 4 benchmark does not rely on a single run or a single metric. It uses repeated seeded splits, fault-specific evaluation, and paired statistical testing. That makes the recommendation more defensible and closer to the standard expected in a formal review.

A third outcome is reproducibility. The data generator is seedable and parameterized, the benchmark writes result tables and a fault ledger, and the repository structure is designed so

another reviewer can follow the work from a clean clone. This is especially important for the internship because the review is based on artifacts, not only verbal explanation.

4. Key Learnings

One key learning is that anomaly detection is highly regime-dependent. The best method depends on whether the goal is low-latency screening, accuracy-maximizing offline review, or high-stakes escalation. A lightweight model can be appropriate in one setting and insufficient in another. This was a useful shift from thinking of the task as "pick the best model" to thinking of it as "choose the right operating path."

Another learning is that synthetic data can be valuable, but only if its limitations are stated clearly. Controlled fault injection makes method comparison possible, but it does not prove real-world deployment performance. The results are best interpreted as a structured benchmark for relative method behavior, not as field validation.

A third learning is that feature design matters as much as model choice. Cross-sensor consistency features, rolling statistics, and fleet-relative z-scores made the benchmark more interpretable. They also made it easier to explain why a method might detect one type of fault better than another.

5. Challenges Encountered

The main challenge was balancing rigor with runtime. The benchmark had to include four methods, five fault types, and 10 seeded splits, while still being practical to run on a local machine. The LSTM autoencoder was the most difficult part because sequence models add training time and sensitivity to preprocessing choices. I handled this by keeping the benchmark window-based and using a compact architecture, but this is still an area that would need more tuning before any production-style claim.

A second challenge was avoiding leakage in the benchmark design. It would have been easy to inject faults first and then split the data, or to let rolling windows carry information across train and test boundaries. I had to be deliberate about splitting before fault injection, training only on clean windows, and fitting scalers only on the training set.

A third challenge was translating technical results into a recommendation. The numerical gap between LOF and One-Class SVM was small in some cases, so the recommendation could not be

based only on a leaderboard. I had to consider operating constraints, false-alert burden, runtime, and interpretability.

6. Closing Reflection

At the midpoint, the internship feels like it has reached the transition point from foundation-building to integration. Weeks 1-2 gave me the product and market context. Weeks 3-4 turned that context into a working telemetry and anomaly detection pipeline. The main improvement area for the second half is to keep the same level of rigor while moving into more abstract tasks such as RL benchmarking, decision-model evaluation, and dashboard design. My goal for Phases C-D is to make the final capstone feel like one connected applied AI story, not just eight weeks of separate deliverables.